

StandardScaler vs RobustScaler

Complete Practical Guide for Machine Learning Preprocessing

Table of Contents

1. Introduction
2. Why Feature Scaling Matters
3. What is StandardScaler?
4. StandardScaler Equation
5. How StandardScaler Works
6. Advantages and Disadvantages of StandardScaler
7. When to Use StandardScaler
8. What is RobustScaler?
9. RobustScaler Equation
10. How RobustScaler Works
11. Advantages and Disadvantages of RobustScaler
12. When to Use RobustScaler
13. StandardScaler vs RobustScaler Comparison
14. Outliers and Their Effect
15. Python Implementation
16. fit(), transform(), and fit_transform()
17. Which Method to Use for X_train, X_test, y_train, and y_test
18. Common Mistakes
19. Best Practices
20. Real-World Example
21. Conclusion

1. Introduction

Feature scaling is one of the most important preprocessing steps in Machine Learning. Many Machine Learning algorithms perform better when numerical features are scaled properly.

Two of the most commonly used scaling techniques are:

- StandardScaler
- RobustScaler

Both methods transform data into a more suitable form for Machine Learning models, but each one is designed for different situations.

2. Why Feature Scaling Matters

Datasets often contain features with different ranges.

Example:

- Age → values between 18 and 60
- Salary → values between 5,000 and 100,000

Without scaling:

- Features with large values dominate the learning process.
- Some algorithms become unstable.
- Optimization becomes slower.

Algorithms highly affected by feature scaling:

- Logistic Regression
 - Support Vector Machine (SVM)
 - K-Nearest Neighbors (KNN)
 - Neural Networks
 - PCA
 - Linear Regression
-

3. What is StandardScaler?

StandardScaler standardizes data by:

- Removing the mean
- Scaling data to unit variance

After transformation:

- Mean becomes approximately 0
- Standard deviation becomes approximately 1

It assumes that the data is approximately normally distributed.

4. StandardScaler Equation

The StandardScaler equation is: $z = \frac{x - \mu}{\sigma}$

Where:

- x = original value
 - μ = mean of the feature
 - σ = standard deviation
-

5. How StandardScaler Works

Step-by-step process:

1. Calculate the mean of the feature.
2. Calculate the standard deviation.
3. Subtract the mean from every value.
4. Divide by the standard deviation.

Result:

- Data becomes centered around zero.
 - Features become comparable.
-

6. Advantages and Disadvantages of StandardScaler

Advantages

- Works very well for normally distributed data
- Improves optimization speed
- Useful for gradient-based algorithms
- Commonly used in ML pipelines

Disadvantages

- Sensitive to outliers
- Extreme values can distort scaling

7. When to Use StandardScaler

Use StandardScaler when:

- Data is approximately normally distributed
- Dataset does not contain many outliers
- Using algorithms sensitive to feature magnitude

Recommended algorithms:

- Logistic Regression
 - SVM
 - PCA
 - Neural Networks
 - Linear Regression
 - KNN
-

8. What is RobustScaler?

RobustScaler is a scaling method designed for datasets containing outliers.

Instead of using:

- Mean
- Standard deviation

It uses:

- Median
- Interquartile Range (IQR)

This makes it more resistant to extreme values.

9. RobustScaler Equation

The RobustScaler equation is: $x_{scaled} = \frac{x - \text{Median}}{IQR}$

Where:

- Median = middle value
 - IQR = Q3 - Q1
 - Q1 = 25th percentile
 - Q3 = 75th percentile
-

10. How RobustScaler Works

Step-by-step process:

1. Calculate the median.
2. Calculate Q1 and Q3.
3. Compute the IQR.
4. Apply scaling transformation.

Result:

- Outliers have less influence.

- Most values remain within a stable range.

11. Advantages and Disadvantages of RobustScaler

Advantages

- Resistant to outliers
- Works well with noisy datasets
- Better for skewed distributions

Disadvantages

- Does not guarantee normal distribution
- Sometimes less effective when data is already clean

12. When to Use RobustScaler

Use RobustScaler when:

- Dataset contains outliers
- Data is heavily skewed
- Real-world data is noisy

Common applications:

- Financial data
- Sensor data
- Real-world business datasets

13. StandardScaler vs RobustScaler Comparison

Feature	StandardScaler	RobustScaler
Uses Mean	Yes	No
Uses Standard Deviation	Yes	No
Uses Median	No	Yes
Resistant to Outliers	No	Yes
Best for Clean Data	Yes	Sometimes
Best for Outliers	No	Yes

14. Outliers and Their Effect

Outliers are extreme values significantly different from other observations.

Example:

- Most salaries between 5,000 and 15,000
- One salary = 1,000,000

Effect on StandardScaler:

- Mean becomes distorted
- Standard deviation becomes very large

Effect on RobustScaler:

- Median remains stable
 - Scaling becomes more reliable
-

15. Python Implementation

StandardScaler Example

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

RobustScaler Example

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

16. fit(), transform(), and fit_transform()

fit()

Learns parameters from the data.

Examples:

- StandardScaler learns mean and standard deviation.
- RobustScaler learns median and IQR.

Example:

```
scaler.fit(X_train)
```

transform()

Uses learned parameters to scale the data.

Example:

```
X_test_scaled = scaler.transform(X_test)
```


fit_transform()

Performs both:

- fit()
- transform()

Together in one step.

Example:

```
X_train_scaled = scaler.fit_transform(X_train)
```

17. Which Method to Use for X_train, X_test, y_train, and y_test

Correct Approach

```
scaler.fit(X_train)

X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

OR

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Why?

The scaler must learn only from training data.

If you use fit_transform() on X_test:

- Information leaks from test data
- Evaluation becomes unrealistic
- This is called Data Leakage

What About y_train and y_test?

Classification Problems

Usually:

- Do NOT scale y_train
- Do NOT scale y_test

Regression Problems

Sometimes target scaling is useful.

Especially when:

- Target values are extremely large
- Using Neural Networks

18. Common Mistakes

Mistake 1

Using `fit_transform()` on `X_test`.

Wrong:

```
X_test_scaled = scaler.fit_transform(X_test)
```

Correct:

```
X_test_scaled = scaler.transform(X_test)
```

Mistake 2

Scaling before train-test split.

Wrong:

```
X_scaled = scaler.fit_transform(X)
```

Correct:

```
X_train, X_test = train_test_split(X, test_size=0.2)
```

19. Best Practices

- Always split data before scaling.
- Fit scaler only on training data.
- Use pipelines when possible.
- Use RobustScaler for noisy datasets.
- Use StandardScaler for clean normal distributions.

20. Real-World Example

Suppose we are building a fraud detection system.

Features:

- Transaction amount
- Customer age
- Account balance

Problem:

Some transactions are extremely large.

Solution:

- StandardScaler may become affected.
- RobustScaler usually performs better.

21. Conclusion

StandardScaler and RobustScaler are essential preprocessing tools in Machine Learning.

Choose:

- StandardScaler for clean and normally distributed data.
- RobustScaler when outliers are present.

Correct preprocessing improves:

- Model performance
- Stability
- Training speed
- Generalization

Understanding when and how to use scaling is a critical Machine Learning skill.